

FORGE PUBLIC SAFETY AWS PLATFORM REBUILD

MASTER CURSOR DEVELOPMENT DIRECTIVE

1. PRIMARY DIRECTIVE

You are rebuilding the existing Forge Academy and Forge RMS applications into a secure, enterprise-grade, multi-tenant platform hosted on Amazon Web Services.

This is not a basic visual redesign, simple Firebase migration, prototype, mockup, or limited proof of concept.

The final system must be:

- Fully operational
- Production-ready
- Secure by default
- Multi-tenant
- Modular
- Scalable
- Mobile-first
- Tablet-friendly
- Accessible
- Highly configurable
- Import-capable
- Export-capable
- Auditable
- API-driven
- Infrastructure-as-code managed
- Deployable in commercial AWS
- Architected for a future independent AWS GovCloud deployment

Preserve all useful functionality, workflows, layouts, terminology, records, integrations, reports, and business logic from the existing Forge Academy and Forge RMS systems.

Do not blindly copy poor Firebase-era architecture.

Retain valuable user-facing functionality while rebuilding the underlying:

- Database
- Authentication
- Authorization
- Tenant isolation

- File storage
- Background processing
- Audit logging
- Configuration system
- Import system
- API layer
- Deployment system
- Monitoring
- Security controls

Do not remove existing features unless they are:

- Duplicated
- Broken
- Obsolete
- Insecure
- Explicitly replaced with a superior implementation

When an existing feature is replaced, preserve its intended business purpose.

2. REQUIRED FIRST ACTIONS

Before writing substantial new application code, perform a complete repository and platform assessment.

Create the following files:

```
/docs/discovery/existing-system-inventory.md
/docs/discovery/firebase-inventory.md
/docs/discovery/feature-preservation-matrix.md
/docs/discovery/data-migration-inventory.md
/docs/discovery/security-gap-analysis.md
/docs/discovery/aws-service-compatibility.md
/docs/discovery/govcloud-readiness-register.md
/docs/discovery/technical-debt-register.md
/docs/discovery/open-decisions.md
```

Inspect and document:

- Existing repositories
- Applications
- Packages
- Routes
- Pages
- Components

- Firebase projects
- Firestore collections
- Firestore subcollections
- Firebase Authentication usage
- Firebase Storage usage
- Cloud Functions
- Scheduled functions
- Security rules
- Environment variables
- API integrations
- Data models
- User roles
- Tenant logic
- Academy modules
- RMS modules
- Reporting features
- Imports
- Exports
- File generation
- PDF generation
- Email systems
- Notification systems
- Mobile workflows
- Offline workflows
- Existing defects
- Existing incomplete features
- Hard-coded configuration
- Hard-coded dropdowns
- Hard-coded tenant information
- Existing user feedback embedded in the codebase

Create a preservation matrix with these statuses:

```
PRESERVE  
PRESERVE_AND_ENHANCE  
REBUILD  
CONSOLIDATE  
DEPRECATE  
REMOVE_AFTER_APPROVAL  
UNKNOWN
```

Do not begin deleting Firebase functionality until the replacement feature has been implemented, tested, and documented.

3. PROJECT ORGANIZATION

Use a monorepo.

Recommended structure:

```
forge-platform/
├─ apps/
│   ├── academy-web/
│   ├── rms-web/
│   ├── creator-console/
│   ├── tenant-admin/
│   ├── department-portal/
│   ├── student-portal/
│   ├── public-registration/
│   ├── api-gateway/
│   ├── academy-api/
│   ├── rms-api/
│   ├── identity-api/
│   ├── configuration-api/
│   ├── import-api/
│   ├── reporting-api/
│   ├── notification-api/
│   ├── integration-api/
│   └─ worker-service/
├─ packages/
│   ├── ui/
│   ├── design-system/
│   ├── auth/
│   ├── authorization/
│   ├── database/
│   ├── validation/
│   ├── configuration/
│   ├── audit/
│   ├── events/
│   ├── imports/
│   ├── exports/
│   ├── reporting/
│   ├── documents/
│   ├── notifications/
│   ├── integrations/
│   ├── observability/
│   └─ security/
```

```
|
| |
| | | shared-types/
| | | api-client/
| | | testing/
| | | └─ govcloud-compatibility/
|
| └─ infrastructure/
|   |
|   | | cdk/
|   | | |
|   | | | bin/
|   | | | lib/
|   | | | └─ constructs/
|   | | | └─ environments/
|   | | | └─ tests/
|   | |
|   | | └─ policies/
|   | | └─ waf/
|   | | └─ monitoring/
|   | | └─ backup/
|   | | └─ disaster-recovery/
|   | | └─ deployment/
|
| └─ database/
|   |
|   | | migrations/
|   | | seeds/
|   | | fixtures/
|   | | views/
|   | | functions/
|   | | └─ row-level-security/
|   | | └─ documentation/
|
| └─ migration/
|   |
|   | | firebase-export/
|   | | transformations/
|   | | validators/
|   | | reconciliation/
|   | | attachment-migration/
|   | | └─ reports/
|
| └─ docs/
|   |
|   | | architecture/
|   | | api/
|   | | data-model/
|   | | security/
|   | | operations/
|   | | deployment/
|   | | migration/
|   | | user-guides/
|   | | └─ decisions/
```

```
|
├─ scripts/
├─ tests/
├─ .github/
│   └─ workflows/
├─ package.json
├─ pnpm-workspace.yaml
├─ turbo.json
└─ README.md
```

Use:

- TypeScript
- React
- Next.js where appropriate
- NestJS or a similarly structured TypeScript backend
- PostgreSQL
- Prisma or Drizzle ORM
- Zod for runtime validation
- pnpm
- Turborepo
- ESLint
- Prettier
- Vitest or Jest
- Playwright
- Docker
- AWS CDK in TypeScript

Do not mix multiple backend frameworks without a documented architectural reason.

4. AWS ARCHITECTURE

4.1 Commercial AWS account structure

Prepare the application for this AWS Organization structure:

```
Forge Commercial AWS Organization
├─ Management
├─ Security
├─ Log Archive
├─ Shared Services
├─ Development
├─ Testing
└─ Staging
```

```
|— Production
|— Migration
└— Disaster Recovery
```

Do not assume all accounts exist during local development.

Support staged deployment so the application can initially deploy into a smaller environment and later expand into the full account structure.

4.2 Future GovCloud structure

Prepare for:

```
Forge GovCloud AWS Organization
|— Management
|— Security
|— Log Archive
|— Shared Services
|— Development
|— Staging
|— Production
└— Disaster Recovery
```

Commercial AWS and GovCloud must remain completely separate.

Never create runtime dependencies from GovCloud to commercial AWS for protected workloads.

4.3 Required AWS services

Use the following services where appropriate:

Application delivery

- Amazon Route 53
- Amazon CloudFront
- Amazon S3
- AWS Certificate Manager
- AWS WAF

Application compute

- Amazon ECS Fargate for primary APIs and sustained application workloads
- AWS Lambda for bounded event-driven workloads
- Amazon API Gateway when appropriate
- Application Load Balancer

- Amazon ECR

Database

- Amazon Aurora PostgreSQL-Compatible Edition
- Amazon RDS Proxy where supported and justified
- Amazon ElastiCache only for clear caching or rate-limiting use cases

Files

- Amazon S3
- S3 versioning
- S3 lifecycle policies
- S3 object lock only where required
- S3 access points where useful
- Pre-signed upload and download URLs

Identity

- Amazon Cognito
- MFA
- SAML support
- OIDC support
- Tenant-aware login routing
- AWS IAM Identity Center for AWS workforce administration

Background processing

- Amazon SQS
- AWS Step Functions
- Amazon EventBridge
- EventBridge Scheduler
- Dead-letter queues

Security

- AWS KMS
- AWS Secrets Manager
- AWS Security Hub
- Amazon GuardDuty
- Amazon Inspector
- Amazon Macie
- AWS Config
- AWS CloudTrail
- AWS Shield where applicable
- AWS WAF

Monitoring

- Amazon CloudWatch
- Structured application logs
- Metrics
- Alarms
- Dashboards
- Distributed request tracing where supported

Communications

- Amazon SES
- Amazon SNS where appropriate
- Provider abstraction for SMS
- Provider abstraction for push notifications

Backup and recovery

- AWS Backup
- Automated database backups
- Cross-account backups
- Recovery testing
- Restore documentation

Do not introduce a commercial AWS service unless it has been checked for GovCloud availability or an abstraction layer has been created.

5. PARTITION-NEUTRAL DEVELOPMENT

The code must support both:

```
aws
aws-us-gov
```

Do not hard-code:

```
arn:aws
amazonaws.com
us-east-1
commercial account IDs
commercial resource IDs
Cognito pool IDs
bucket names
database endpoints
```

KMS key ARNs
CloudFront domains
API endpoints

Create centralized environment configuration.

Required values include:

```
APP_ENV
AWS_PARTITION
AWS_REGION
AWS_ACCOUNT_ID
AWS_SECONDARY_REGION
DATABASE_URL
DATABASE_SECRET_ARN
COGNITO_USER_POOL_ID
COGNITO_CLIENT_ID
COGNITO_DOMAIN
KMS_GENERAL_KEY_ARN
KMS_SENSITIVE_DATA_KEY_ARN
S3_DOCUMENT_BUCKET
S3_IMPORT_BUCKET
S3_EXPORT_BUCKET
S3_AUDIT_BUCKET
SQS_IMPORT_QUEUE_URL
SQS_NOTIFICATION_QUEUE_URL
SES_FROM_ADDRESS
PUBLIC_APP_URL
API_BASE_URL
LOG_LEVEL
FEATURE_FLAG_PROVIDER
```

Create a partition utility:

```
export interface AwsPartitionConfiguration {
  partition: "aws" | "aws-us-gov";
  dnsSuffix: string;
  arnPrefix: string;
  region: string;
}
```

No application package should independently calculate AWS partition behavior.

6. ENVIRONMENTS

Support:

```
local
development
testing
staging
production
govcloud-development
govcloud-staging
govcloud-production
```

Each environment must use:

- Separate secrets
- Separate databases
- Separate Cognito resources
- Separate S3 buckets
- Separate encryption keys
- Separate logs
- Separate deployment approvals
- Separate API URLs
- Separate email settings

Development and testing environments must use synthetic or sanitized data only.

Do not use real Social Security numbers in:

- Local development
- Automated tests
- Demo accounts
- Screenshots
- Seed files
- Fixtures
- Error reports

7. CORE PLATFORM DATA MODEL

Create a detailed database design in:

```
/docs/data-model/core-platform-data-model.md
```

Use UUIDs for internal identifiers.

Recommended core tables:

```
tenants
tenant_settings
tenant_domains
tenant_entitlements
tenant_features
tenant_branding
tenant_security_settings

organizations
organization_types
organization_relationships
organization_addresses
organization_contacts

persons
person_names
person_addresses
person_contacts
person_identifiers
person_emergency_contacts
person_organization_memberships
person_merge_history
person_restrictions

users
user_identities
user_tenant_memberships
user_sessions
user_security_events
user_preferences

roles
permissions
role_permissions
user_role_assignments
resource_access_rules

products
modules
```

features
subscription_plans
subscriptions
subscription_events

dropdown_definitions
dropdown_options
dropdown_dependencies
dropdown_aliases
dropdown_versions

custom_field_definitions
custom_field_options
custom_field_values
custom_field_versions

form_definitions
form_versions
form_sections
form_fields
form_submissions
form_submission_values

workflow_definitions
workflow_versions
workflow_states
workflow_transitions
workflow_actions
workflow_instances
workflow_events

documents
document_versions
document_links
document_access_events
document_retention_rules

notifications
notification_templates
notification_deliveries
notification_preferences

import_jobs
import_files
import_profiles
import_column_mappings
import_rows
import_row_errors

```
import_duplicate_candidates
import_batches
import_rollback_events

export_jobs
export_files
export_access_events

audit_events
security_events
support_access_sessions

integration_definitions
integration_credentials
integration_events
webhook_subscriptions
webhook_deliveries

feature_flags
system_settings
release_versions
```

Every tenant-owned table must include:

```
tenant_id
created_at
created_by
updated_at
updated_by
version
```

Where appropriate, include:

```
deleted_at
deleted_by
archived_at
archived_by
effective_at
expires_at
```

Use soft deletion for records that may be referenced historically.

Do not permanently delete referenced dropdown values, certifications, courses, forms, workflows, or statuses.

8. TENANT ISOLATION

Implement tenant isolation at all layers.

8.1 API layer

Every request must resolve:

- Authenticated user
- Active tenant
- Product access
- Module access
- Role assignments
- Permission set
- Resource scope

Never trust a tenant ID received from the frontend without validating it against the authenticated user.

8.2 Database layer

Use PostgreSQL Row-Level Security.

Create session-level tenant context, such as:

```
SET LOCAL app.current_tenant_id = 'tenant-uuid';  
SET LOCAL app.current_user_id = 'user-uuid';
```

Create RLS policies for tenant-owned tables.

Example pattern:

```
ALTER TABLE students ENABLE ROW LEVEL SECURITY;  
  
CREATE POLICY tenant_isolation_students  
ON students  
USING (tenant_id = current_setting('app.current_tenant_id')::uuid)  
WITH CHECK (tenant_id = current_setting('app.current_tenant_id')::uuid);
```

Do not permit application database roles to bypass RLS.

Create automated tests that attempt cross-tenant reads and writes.

8.3 Storage layer

S3 object keys must include tenant scope.

Example:

```
tenants/{tenantId}/documents/{documentId}/{versionId}
tenants/{tenantId}/imports/{importJobId}/{filename}
tenants/{tenantId}/exports/{exportJobId}/{filename}
```

Do not expose raw bucket access.

Use pre-signed URLs with:

- Short expiration
- Tenant validation
- User validation
- Permission validation
- File-purpose validation
- Audit logging

8.4 Background jobs

Every queue message must include:

```
interface TenantJobContext {
    tenantId: string;
    initiatedByUserId?: string;
    correlationId: string;
    jobType: string;
    resourceId?: string;
}
```

Workers must reject jobs missing tenant context.

9. AUTHENTICATION AND AUTHORIZATION

9.1 Authentication

Use Amazon Cognito for customer authentication.

Support:

- Username and password
- Email login
- MFA
- Password reset
- Account recovery
- Tenant invitation
- Department invitation
- Student self-registration
- SAML
- OIDC
- Future government identity providers
- Session expiration
- Forced sign-out
- Account disablement
- Risk-based controls where available

9.2 Authorization

Do not use Cognito groups as the complete authorization system.

Implement application-level RBAC with optional resource conditions.

Permission examples:

```
tenant.view
tenant.configure
tenant.branding.manage

person.view
person.create
person.edit
person.archive
person.merge
person.view_sensitive
person.view_ssn
person.export_sensitive

student.view
student.create
student.edit
student.enroll
student.withdraw
student.complete
```

instructor.view
instructor.assign
instructor.qualifications.manage

course.view
course.create
course.edit
course.publish
course.archive

class.view
class.create
class.schedule
class.publish
class.cancel
class.complete

exam.author
exam.publish
exam.administer
exam.grade
exam.override

skill.evaluate
skill.override
skill.remediate

certificate.issue
certificate.revoke
certificate.reissue

finance.view
finance.invoice
finance.payment.record
finance.refund
finance.export

rms.personnel.manage
rms.training.manage
rms.schedule.manage
rms.apparatus.manage
rms.inventory.manage
rms.inspection.manage
rms.incident.manage

import.view
import.upload
import.map

```
import.validate
import.execute
import.approve
import.rollback

export.standard
export.sensitive

configuration.dropdown.manage
configuration.custom_fields.manage
configuration.forms.manage
configuration.workflows.manage

audit.view
security.view
support.authorize
```

Create default roles, but allow tenants to duplicate and customize them.

Do not allow tenant administrators to grant platform-creator permissions.

10. SENSITIVE INFORMATION

10.1 Social Security numbers

Social Security numbers must be optional and isolated.

Do not store SSNs in ordinary person-profile columns.

Use a dedicated restricted structure:

```
person_sensitive_identifiers
```

Recommended fields:

```
id
tenant_id
person_id
identifier_type
encrypted_value
last_four
key_version
```

```
verification_status
created_at
created_by
updated_at
updated_by
```

Required SSN behavior:

- Encrypt before database persistence
- Use a dedicated KMS key
- Store only encrypted value and last four digits
- Never use SSN as a primary key
- Never use SSN in URLs
- Never write SSN to logs
- Never include SSN in analytics
- Never include full SSN in routine API responses
- Never include full SSN in normal exports
- Mask as `***-**-1234`
- Require explicit `person.view_ssn` permission
- Require a reason before revealing
- Require recent MFA or step-up authentication
- Audit each full-value access
- Rate-limit reveal attempts
- Automatically re-mask after a short period
- Prevent browser caching where practical
- Prevent accidental clipboard-oriented UI
- Watermark reports containing SSNs
- Restrict printing
- Provide retention and deletion controls

Create:

```
POST /persons/:personId/sensitive-identifiers/:identifierId/reveal
```

The request must include:

```
{
  "reason": "Required for authorized state enrollment verification"
}
```

The response must be short-lived and must not be cached.

10.2 FEMA SID

Store FEMA SID as a protected external identifier.

Support:

- Uniqueness rules
- Duplicate detection
- Verification status
- Masked display where appropriate
- Restricted exports
- Import validation
- Correction history
- Audit events

Do not use FEMA SID as an internal foreign key.

10.3 Additional protected data

Apply heightened controls to:

- Date of birth
- Driver's license number
- Home address
- Personal telephone number
- Personal email
- Emergency contacts
- Background-check information
- Medical-clearance information
- Disciplinary information
- Banking information
- Payment information
- Exam records
- Personnel evaluations

Separate access permissions by category.

11. SHARED PERSON IDENTITY SYSTEM

Create one immutable `ForgePersonId`.

A person may exist across multiple tenants without those tenants automatically sharing the same complete record.

Separate:

Global identity resolution
Tenant-owned person profile
Organization membership
Product-specific record

Support:

- Student record
- Instructor record
- Department employee record
- Academy employee record
- RMS personnel record
- Evaluator record
- Department contact
- Billing contact

Create duplicate matching based on configurable rules.

Possible matching identifiers:

- Existing ForgePersonId
- FEMA SID
- External system ID
- Employee number
- Student number
- Email address
- Name plus date of birth
- Name plus department

Do not automatically merge people based on name alone.

All merges must:

- Be authorized
- Show both records
- Show conflicts
- Preserve source IDs
- Preserve an audit history
- Support reversal where practical

12. CONFIGURATION STUDIO

Build a complete tenant Configuration Studio.

It must not be a collection of placeholder screens.

Authorized users must be able to manage:

- Tenant profile
- Academy profile
- Department profile
- Branding
- Product terminology
- Navigation
- Modules
- Features
- Dropdowns
- Custom fields
- Forms
- Workflows
- Roles
- Permissions
- Notifications
- Email templates
- Document templates
- Certificate templates
- Numbering formats
- Reporting settings
- Dashboards
- Imports
- Exports
- Integrations
- Security settings
- Retention settings
- Business hours
- Holidays
- Facilities
- Locations

12.1 Terminology

Allow tenants to change display terminology.

Examples:

Student → Recruit
Class → Course Offering
Department → Agency

Instructor → Faculty
Dormitory → Residence Hall

System keys must remain stable.

Only display labels should change.

12.2 Navigation editor

Allow authorized tenant administrators to:

- Rename menu items
- Reorder modules
- Hide permitted modules
- Group related items
- Configure collapsible sections
- Set role visibility
- Set product visibility
- Restore defaults

Core security and account controls must not be removable.

12.3 Configuration versioning

Every configuration object must support:

- Draft
- Published
- Scheduled
- Archived
- Version history
- Comparison
- Rollback
- Effective date

Do not allow configuration changes to corrupt historical records.

13. DROPDOWN AND REFERENCE DATA SYSTEM

Do not hard-code tenant-controlled dropdown values in React components.

Create a central dropdown service.

Dropdown definitions must include:

```
key
system_name
display_name
description
module
record_type
selection_type
tenant_editable
system_locked
supports_color
supports_effective_dates
supports_dependencies
supports_aliases
```

Dropdown options must include:

```
code
label
description
sort_order
is_default
is_active
color_token
effective_at
expires_at
metadata
```

Support:

- Single select
- Multi-select
- Searchable select
- Grouped options
- Dependent dropdowns
- Hierarchical options
- Effective dates
- Role restrictions
- Import aliases
- Module restrictions
- Conditional visibility

Examples:

- Rank
- Employment type

- Department type
- Student status
- Enrollment status
- Course category
- Course type
- Delivery method
- Attendance status
- Absence reason
- Certification type
- Certification level
- Certification status
- Payment method
- Payment status
- Exam status
- Skill result
- Remediation reason
- Building
- Room
- Shift
- Apparatus type
- Inspection type
- Violation category
- Hydrant status
- Incident category

When an option is deactivated:

- Preserve it on historical records
- Hide it from new selection
- Allow restoration
- Do not delete it when referenced

Provide import and export for dropdown values.

14. CUSTOM FIELD BUILDER

Support custom fields on:

- Person
- Student
- Instructor
- Department
- Academy
- Course
- Class
- Enrollment

- Attendance
- Certification
- Exam
- Skill evaluation
- Payment
- Invoice
- Dorm assignment
- Facility
- Apparatus
- Equipment
- Inspection
- Occupancy
- Incident
- Document

Field types:

- Short text
- Long text
- Integer
- Decimal
- Currency
- Percentage
- Date
- Time
- Date and time
- Boolean
- Checkbox
- Radio group
- Single-select
- Multi-select
- Email
- Phone
- URL
- Address
- File
- Signature
- Person lookup
- Organization lookup
- Department lookup
- Course lookup
- Apparatus lookup
- Calculated field

Support:

- Required
- Optional

- Default values
- Help text
- Validation
- Character limits
- Number limits
- Regular expression validation
- Conditional visibility
- Role visibility
- Role editability
- Import mapping
- Reporting availability
- API availability
- Versioning
- Effective dates

Do not allow unrestricted custom code execution.

15. FORM BUILDER

Create a drag-and-drop form builder.

Support:

- Form title
- Description
- Sections
- Columns
- Standard fields
- Custom fields
- Instructions
- Conditional logic
- Validation
- Signatures
- File attachments
- Approval steps
- Draft saving
- Submission locking
- Mobile layout
- Printable output
- Versioning
- Preview
- Scheduled publishing
- Retirement
- Duplication

Form records must retain:

```
form_definition_id  
form_version_id  
submission_snapshot
```

Historical submissions must remain renderable after the form changes.

16. WORKFLOW BUILDER

Create a no-code workflow builder.

Each workflow can include:

- States
- Transitions
- Conditions
- Required permissions
- Required fields
- Assignments
- Notifications
- Timers
- Escalations
- Approvals
- Rejections
- Reopening
- Completion
- Automatic actions

Examples:

```
Student Application  
Draft  
Submitted  
Under Review  
Additional Information Required  
Approved  
Denied  
Waitlisted  
Enrolled  
Withdrawn  
Completed
```

Inspection
Draft
Scheduled
Assigned
In Progress
Submitted
Violations Open
Reinspection Required
Approved
Closed
Reopened

Apparatus Repair
Reported
Reviewed
Assigned
Awaiting Parts
Repair In Progress
Ready for Verification
Returned to Service
Closed

Workflow actions may:

- Send email
- Create notification
- Assign task
- Generate document
- Update status
- Set due date
- Create audit event
- Call an approved webhook
- Queue a background job

Implement workflow versioning.

17. UNIVERSAL IMPORT CENTER

The Import Center is a foundation feature.

Do not build separate unrelated import logic inside each module.

Create a shared import engine with module-specific schemas.

17.1 Supported formats

- CSV
- XLSX
- JSON
- ZIP migration bundles
- API import
- SFTP import where approved

17.2 Required workflow

1. Select product.
2. Select module.
3. Select record category.
4. Download a template.
5. Upload the source file.
6. Malware-scan the file.
7. Detect worksheet and header row.
8. Detect source columns.
9. Map columns.
10. Apply transformations.
11. Map dropdown values.
12. Apply defaults.
13. Validate records.
14. Detect duplicates.
15. Preview proposed actions.
16. Save the mapping profile.
17. Run a test import.
18. Review errors and warnings.
19. Approve production import.
20. Queue processing.
21. Display progress.
22. Generate a result report.
23. Download rejected rows.
24. Correct and reprocess failed rows.
25. Roll back where safe.

17.3 Import row statuses

UPLOADED
PENDING_VALIDATION
READY
WARNING
ERROR
DUPLICATE
SKIPPED

```
CREATE
UPDATE
MERGE_REVIEW
CREATED
UPDATED
FAILED
ROLLED_BACK
```

17.4 Validation

Validate:

- Required fields
- Invalid dates
- Invalid emails
- Invalid telephone numbers
- Invalid identifiers
- Duplicate FEMA SID
- Duplicate employee numbers
- Duplicate student numbers
- Unknown dropdown values
- Inactive dropdown values
- Missing parent records
- Invalid relationships
- Invalid certification periods
- Invalid workflow states
- Cross-tenant references
- Unauthorized fields
- Invalid currency
- Invalid status transitions
- Invalid file references

Never silently discard invalid rows.

17.5 Duplicate handling

Support:

- Create new
- Update existing
- Skip
- Manual review
- Merge candidate
- Reject

Provide confidence levels for automated matching.

17.6 Saved import profiles

Store:

- Profile name
- Source system
- Product
- Module
- Record category
- Column mappings
- Date formats
- Defaults
- Transformations
- Dropdown aliases
- Duplicate rules
- Permissions
- Version
- Last used

17.7 Background processing

Use:

- S3 upload
- SQS queue
- Step Functions
- Lambda or ECS worker
- Dead-letter queue
- Progress events
- Idempotent processing

The browser must not need to remain open.

17.8 Import rollback

Rollback must:

- Be permission controlled
 - Be available only when safe
 - Show affected records
 - Refuse rollback when later dependent changes would be corrupted
 - Record a complete audit event
-

17A. SHARED QR CODE AND PUBLIC ACCESS PLATFORM

The QR platform is a shared Forge foundation capability. Forge Industrial Safety is the first production consumer, but the shared core must remain product-neutral and reusable by Forge RMS, Forge Academy, and future Forge products.

17A.1 Required outcomes

- Generate fully functional QR codes for stored PDF files, procedures, documents, forms, equipment records, facilities, locations, and other approved targets.
- Allow authorized users to create, edit, activate, disable, archive, restore, revoke, soft-delete, duplicate, download, print, and batch-generate QR links.
- Allow a QR destination to be replaced or versioned without requiring the printed QR code to be replaced.
- Provide controlled anonymous public access where explicitly enabled, while preserving tenant isolation, auditing, rate limiting, and access-policy enforcement.
- Provide SVG, PNG, and printable PDF label output with decode verification before release.
- Provide scan analytics without collecting unnecessary personal information.

17A.2 Dynamic resolver requirement

Do not encode raw S3 URLs, internal API routes, tenant IDs, record IDs, database IDs, bucket names, or infrastructure identifiers into QR images. Every QR code must resolve through a configurable Forge public resolver.

```
https://go.forgepublicsafety.com/q/{publicToken}
```

The public token must be cryptographically secure, revocable, rotatable, non-sequential, and unrelated to internal identifiers. The resolver domain must be configuration-driven so commercial AWS, future GovCloud, and product-specific deployments can use separate domains without changing the shared code.

17A.3 Supported targets

- Managed documents and document versions
- PDF files, policies, SOPs, SOGs, manuals, safety procedures, work instructions, SDS files, certificates, reports, notices, and training documents
- Forms and public form submissions where anonymous access is approved
- Equipment, machinery, assets, apparatus, inventory items, facilities, buildings, rooms, areas, locations, hydrants, occupancies, preplans, inspections, courses, classes, and training records
- External HTTPS destinations only when the tenant has `qr.external_link.manage` permission and the destination passes allowlist, validation, and open-redirect controls
- Public verification pages for certificates, transcripts, credentials, and approved records

Target resolution must use adapters. Product-specific domain packages must not be imported into the shared QR core.

17A.4 Shared data model

Add the following shared tables to the core platform data model:

- qr_links
- qr_link_versions
- qr_link_targets
- qr_link_access_policies
- qr_link_scan_events
- qr_link_download_events
- qr_link_audit_events
- qr_link_print_templates
- qr_link_batches
- qr_link_labels
- public_access_sessions
- public_access_acknowledgments

Every tenant-owned QR table must include tenant_id, created_at, created_by, updated_at, updated_by, and version. Add archived_at, archived_by, deleted_at, deleted_by, effective_at, expires_at, revoked_at, and revoked_by where applicable.

Required OR status model:

DRAFT, SCHEDULED, ACTIVE, DISABLED, EXPIRED, ARCHIVED, REVOKED, DELETED

Use PostgreSQL row-level security with FORCE ROW LEVEL SECURITY on every tenant-owned QR table. Public resolver access must use a narrowly scoped service path that resolves only the public token and approved published target; it must not bypass tenant isolation for general database access.

17A.5 Permissions

- qr.view
- qr.create
- qr.edit
- qr.archive
- qr.restore
- qr.delete
- qr.activate
- qr.disable
- qr.revoke
- qr.public_access.manage
- qr.destination.manage
- qr.download
- qr.print
- qr.batch.create
- qr.analytics.view
- qr.audit.view

- qr.template.manage
- qr.external_link.manage

Tenant administrators may grant tenant-scoped QR permissions only. They must not grant creator-level platform administration or cross-tenant access.

17A.6 Public access policies

An authorized user may configure:

- Anonymous open access
- Activation and expiration dates
- Maximum successful scans or downloads
- Password or PIN protection
- Required acknowledgment
- Optional terms acceptance
- IP or network restrictions where justified
- Geographic restrictions where justified
- Download allowed or view-only behavior
- Published-version-only behavior
- Emergency access behavior where contractually authorized

The resolver must fail closed. Disabled, archived, expired, revoked, deleted, unknown, malformed, over-limit, or policy-denied tokens must not reveal the destination or tenant identity.

17A.7 Document and storage integration

- A QR link may target the latest published document version or an explicitly pinned version.
- Replacing a PDF or publishing a new procedure version must not change the printed QR image when the link is configured for latest published version.
- S3 objects must remain tenant-scoped, encrypted, versioned, malware-scanned, and accessible only through short-lived signed delivery after resolver authorization.
- Raw S3 URLs must never be returned as permanent public links.
- Document retention, legal hold, archive, and revocation rules must be honored.
- Public downloads and restricted views must create auditable access events.

17A.8 QR image and label generation

- Generate standards-compliant QR images in SVG and PNG.
- Generate printable PDF label sheets and individual labels.
- Support tenant branding, approved logos, readable titles, asset identifiers, instructions, and configurable label templates.
- Support common Avery layouts, thermal printers, equipment tags, apparatus labels, procedure labels, and batch print runs.
- Maintain adequate quiet zones and error correction.
- Decode-test every generated QR image before marking it ready for production download or printing.
- Store generation metadata, template version, encoded resolver URL, checksum, and decode-verification result.

17A.9 Shared APIs

Provide versioned APIs under `/api/v1/qr-links` and a separate public resolver boundary. Required operations include:

- Create, read, search, filter, paginate, update, duplicate, archive, restore, revoke, disable, activate, and soft-delete links
- Assign and change destinations
- Configure and validate public access policies
- Generate and download SVG, PNG, and PDF labels
- Create and monitor batches
- Retrieve scan analytics and audit history
- Rotate or revoke public tokens
- Resolve public tokens and deliver approved content or landing pages

Every management endpoint must enforce authentication, tenant resolution, entitlements, permission checks, input and output validation, correlation IDs, structured logging, and audit events.

17A.10 Management interfaces

Create a shared QR Management Center available through the Forge Creator Console and authorized tenant administration interfaces.

- Dashboard
- All QR codes
- Create QR code
- Documents and PDFs
- Procedures
- Equipment and assets
- Forms and inspections
- Batch generator
- Print labels
- Scan analytics
- Archived and revoked links
- Templates
- Configuration and access policies

The interface must include search, filters, sorting, pagination, bulk actions, status indicators, destination health, expiration warnings, failed-scan visibility, mobile layout, loading states, empty states, and accessible error handling.

17A.11 Forge Industrial first-consumer adapter

The initial Forge Industrial implementation must support adapters for:

- Safety procedures and policies
- Lockout/tagout procedures and equipment-specific LOTO procedures
- Equipment, machinery, assets, facilities, buildings, areas, and locations
- Safety Data Sheets
- Job Safety Analyses
- Inspections and corrective actions
- Forms
- Emergency response plans
- Training documents and work instructions
- Confined-space, hot-work, fall-protection, electrical-safety, and contractor-safety procedures

Forge Industrial is the first consumer, not the owner of the shared architecture. Do not import Industrial domain packages into the shared QR core.

17A.12 Security, privacy, and observability

- Use cryptographically secure random public tokens and constant-time token comparison where appropriate.
- Apply AWS WAF, resolver rate limiting, abuse detection, bot controls where justified, and structured security events.
- Prevent open redirects, path traversal, header injection, script injection, content-type confusion, and cache leakage.
- Do not place sensitive data in public URLs, logs, analytics, QR titles, or label metadata.

- Record scans using minimal data: timestamp, result, link/version, user agent category, approximate region only when approved, and correlation ID.
- Support token revocation and emergency disablement without deleting historical audit records.
- Provide CloudWatch metrics and alarms for resolver errors, denied scans, malware failures, excessive scanning, and destination failures.

17A.13 Accessibility and testing

- Target WCAG 2.2 AA for management and public landing pages.
- Provide keyboard navigation, semantic markup, screen-reader labels, visible focus, accessible errors, sufficient contrast, and non-color-only statuses.
- Test QR decoding at multiple sizes and print resolutions.
- Test tenant isolation, authorization, public-policy enforcement, expiration, revocation, password protection, scan limits, destination replacement, document versioning, batch generation, and malformed tokens.
- Test that Tenant A cannot view, alter, print, analyze, or resolve Tenant B private QR links.
- Test that public resolver access cannot be used to enumerate tenants or internal record identifiers.
- Test signed delivery expiration and denial after archive, revoke, disable, delete, or document withdrawal.

17A.14 Definition of done

The QR platform is complete only when it includes working UI, APIs, migrations, RLS, permissions, audit events, dynamic resolver, document delivery, image generation, label generation, batch processing, search, filters, archive/restore/revoke, public-access controls, analytics, monitoring, automated tests, deployment documentation, rollback instructions, and product-neutral adapter contracts.

Do not mark the platform complete while it contains placeholder buttons, mock-only data, direct S3 links, hard-coded tenant values, unverified QR images, unprotected public routes, missing audit events, or unresolved cross-tenant exposure.

18. EXPORT CENTER

Create a shared export engine.

Support:

- CSV
- XLSX
- PDF
- JSON where permitted
- ZIP package
- Scheduled exports

Exports must honor:

- Tenant scope
- Product permissions
- Module permissions
- Field permissions
- Sensitive-data restrictions
- Retention policy
- Watermark rules

Sensitive exports must require:

- Additional permission
- Purpose
- Confirmation
- Audit event
- Short-lived download link
- Optional approval

19. FORGE ACADEMY MODULES

Build all modules as production features with:

- Permissions
- Imports
- Exports
- Audit events
- Search
- Filters
- Reports
- Mobile support
- Configuration support

19.1 Academy dashboard

Include:

- Active students
- Upcoming classes
- Open seats
- Waitlisted students
- Enrollment trends
- Instructor availability
- Certification expirations
- Dorm occupancy
- Unpaid invoices
- Application backlog
- Testing performance
- Import failures
- System alerts

Allow dashboard customization by role.

19.2 Academy administration

Include:

- Academy profile
- Campuses
- Locations
- Buildings
- Classrooms
- Facilities
- Departments
- Partner agencies
- Academic terms
- Fiscal years
- Holidays
- Calendars
- Equipment
- Documents
- Configuration Studio
- Audit Center

19.3 Student information

Include:

- ForgePersonId
- Student ID

- FEMA SID
- Protected SSN
- Legal name
- Preferred name
- Contact details
- Department affiliations
- Employment status
- Training history
- Certifications
- Transcript
- Enrollment history
- Documents
- Emergency contacts
- Restrictions
- Accommodations
- Portal access
- Notes with permissions
- Communication history
- Duplicate resolution

19.4 Admissions and registration

Include:

- Public catalog
- Applications
- Department submissions
- Self-registration
- Prerequisite checks
- Approval routing
- Waitlists
- Reserved seats
- Registration limits
- Deadlines
- Transfers
- Substitutions
- Withdrawals
- Cancellations
- Required documents
- Waivers
- Digital signatures
- Payment requirements
- Automated notifications

19.5 Courses

Include:

- Course code
- Course title
- Description
- Category
- Type
- Delivery method
- Prerequisites
- Equivalencies
- Contact hours
- Credit hours
- Objectives
- Curriculum
- Required equipment
- Required materials
- Certification outcomes
- Version history
- Approval
- Archive

19.6 Classes

Include:

- Course version
- Start date
- End date
- Sessions
- Capacity
- Reserved seats
- Campus
- Classroom
- Instructors
- Evaluators
- Roster
- Attendance
- Materials
- Announcements
- Assignments
- Status
- Completion rules
- Cancellation
- Rescheduling
- Waitlist

19.7 Instructor management

Include:

- Instructor profile
- Application
- Approval
- Qualifications
- Certifications
- Authorized courses
- Availability
- Assignments
- Workload
- Evaluations
- Continuing education
- Expiration alerts
- Contracts
- Documents
- Payroll-support export

19.8 LMS

Include:

- Course modules
- Lessons
- Video
- Files
- Reading assignments
- Quizzes
- Assignments
- Completion tracking
- Discussions if enabled
- Prerequisites
- Release dates
- Due dates
- Accessibility
- Progress
- Instructor feedback

19.9 Testing

Include:

- Question banks
- Categories
- Difficulty

- Randomization
- Exam versions
- Timers
- Attempt limits
- Passing score
- Auto-grading
- Manual grading
- Essay questions
- Retesting
- Remediation
- Appeals
- Exam security
- Item analysis
- Test analytics
- Audit history

19.10 Skills and evaluations

Include:

- Skills sheets
- Stations
- Tasks
- Critical steps
- Pass/fail
- Point scoring
- Evaluator assignments
- Candidate rotation
- Comments
- Signatures
- Remediation
- Retesting
- Appeals
- Printable reports
- Offline evaluation

19.11 Attendance

Include:

- Session attendance
- Daily attendance
- QR check-in
- Manual entry
- Late arrival
- Early departure
- Absence reason

- Excused status
- Instructor approval
- Corrections
- Audit history
- Attendance percentage
- Completion impact

19.12 Certifications

Include:

- Certification type
- Level
- Issuing authority
- Number
- Issue date
- Expiration date
- Verification
- Renewal
- Equivalency
- Suspension
- Revocation
- Document
- Alerts
- Academy and RMS synchronization

19.13 Certificates and transcripts

Include:

- Templates
- Numbering
- Digital signature
- QR verification
- Issue approval
- Reissue
- Revocation
- Expiration
- Official transcript
- Unofficial transcript
- Transfer credit
- Course equivalency
- Watermarking
- Public verification

19.14 Dormitory operations

Include:

- Buildings
- Floors
- Rooms
- Beds
- Capacity
- Assignment restrictions
- Two-person rooms
- Check-in
- Check-out
- Room changes
- Conflicts
- Damage reports
- Housekeeping
- Maintenance
- Emergency accountability
- Visitor records
- Meal plans

19.15 Finance

Include:

- Tuition
- Fees
- Department billing
- Individual billing
- Purchase orders
- Invoices
- Payments
- Credits
- Refunds
- Scholarships
- Funding sources
- Sponsorships
- Aging
- Statements
- Financial reports
- Exports
- Payment processor abstraction

Do not store raw card data.

19.16 Department portal

Departments must be able to:

- Maintain roster
 - Submit students
 - Approve applications
 - View class availability
 - Pay invoices
 - View training records
 - Download transcripts
 - Track certifications
 - Receive expiration alerts
 - Manage department contacts
 - Review enrollment status
 - Synchronize permitted information with Forge RMS
-

20. FORGE RMS MODULES

20.1 RMS dashboard

Include:

- Staffing
- Personnel status
- Certification expirations
- Apparatus status
- Open maintenance
- Inventory alerts
- Training deficiencies
- Upcoming training
- Open inspections
- Violations
- Hydrant damage
- Incident activity
- Approval queues
- Notifications

20.2 Personnel

Include:

- Person profile
- Employment record

- Rank history
- Station history
- Shift history
- Assignment
- Certifications
- Training
- Equipment issue
- Awards
- Evaluations
- Restricted disciplinary records
- Emergency contacts
- Leave
- Separation
- Archive
- Academy synchronization

20.3 Training

Include:

- Training calendar
- Drill creation
- Course library
- Objectives
- Attendance
- Instructors
- Evaluations
- Certifications
- Expiration alerts
- ISO reporting
- State reporting
- Academy sync
- QR attendance
- Offline attendance
- Reports

20.4 Scheduling

Include:

- Shift calendar
- Minimum staffing
- Qualifications
- Leave
- Overtime
- Shift trades
- Callbacks

- Apparatus assignments
- Daily roster
- Vacancy alerts
- Payroll export
- Configurable rules
- Approval workflows

20.5 Apparatus and fleet

Include:

- Apparatus profile
- Assignment
- Specifications
- Daily checks
- Weekly checks
- Monthly checks
- Maintenance
- Work orders
- Fuel
- Mileage
- Engine hours
- Parts
- Vendors
- Service history
- Out-of-service status
- Replacement planning
- Documents
- Inspections
- Defects
- Costs

20.6 Inventory

Include:

- Item catalog
- Stations
- Apparatus inventory
- Warehouses
- Issue
- Return
- Transfers
- Stock levels
- Reorder points
- Lot numbers
- Expiration

- PPE
- SCBA
- Hose
- Radios
- Tools
- Medical supplies
- QR codes
- Barcodes
- Purchase orders
- Audits

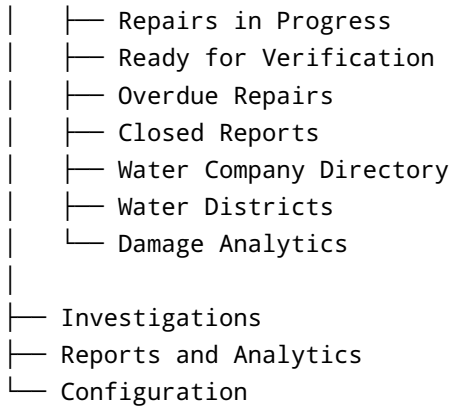
20.7 Prevention navigation

Use:

```

Prevention
├── Inspections
│   ├── Dashboard
│   ├── Work Queue
│   ├── All Inspections
│   ├── My Inspections
│   └── Inspection Reports
├── Occupancies and Preplans
│   ├── Occupancies
│   ├── New Preplan
│   ├── Preplan Work Queue
│   ├── All Preplans
│   └── Offline and Sync
├── Violations and Enforcement
│   ├── Open Violations
│   ├── Notices and Certificates
│   └── Enforcement
├── Permits and Systems
│   ├── Fire Protection Systems
│   ├── Permits
│   ├── Fees
│   └── Payments
├── Hydrants and Water Companies
│   ├── Hydrants
│   ├── Damage Reports
│   ├── Open Damage Reports
│   └── Awaiting Water Company

```



20.8 Inspections

Include:

- Scheduling
- Assignment
- Checklists
- Custom forms
- Offline inspection
- Photos
- Documents
- Signatures
- Violations
- Notices
- Fees
- Reinspection
- Approval
- Closure
- Reporting
- Audit history

20.9 Occupancies and preplans

Include:

- Occupancy profile
- Contacts
- Hazards
- Construction
- Fire protection systems
- Utilities
- Access points
- Knox box
- Floor plans

- Photos
- Apparatus access
- Hydrants
- Tactical notes
- Preplan approval
- Versioning
- Offline use
- Mapping

20.10 Hydrants

Include:

- Location
- Water company
- Water district
- Ownership
- Inspection
- Flow test
- Damage report
- Photos
- Status
- Repair workflow
- Water company email
- Verification
- Overdue alerts
- Map
- Analytics
- Import
- Export

20.11 Incident reporting

Include:

- CAD import
- Auto-population
- NERIS-aligned data model
- Real-time validation
- Mobile
- Offline
- Multi-discipline
- Narrative builder
- Officer review
- Approval
- Property linking
- Occupancy linking

- Personnel assignment
- Apparatus assignment
- Weather
- Wildland support
- Data-quality dashboard
- Annual reports
- Heat maps
- Ad hoc reporting

20.12 EMS operations

Include:

- EMS staffing
- Credential monitoring
- Unit utilization
- Medication inventory
- Controlled-substance checks
- QA/QI
- Supply checks
- Exposure records
- Training requirements

Do not build a full ePCR unless separately authorized.

21. ACADEMY AND RMS INTEGRATION

Create a secure integration boundary.

Do not directly query Academy tables from RMS frontend code.

Use domain services and APIs.

Potentially shared data:

- Person identity
- Department roster
- Course enrollment
- Course completion
- Training history
- Certifications
- Expiration dates
- Instructor qualification
- Registration status

Do not automatically share:

- SSN
- Medical records
- Disciplinary records
- Banking information
- Restricted examination records
- Unrelated tenant data

Every synchronization event must record:

```
source_tenant_id
destination_tenant_id
source_product
destination_product
person_id
data_category
fields_shared
authorization_basis
initiated_by
timestamp
result
correlation_id
```

Support tenant-configurable sharing agreements.

22. CREATOR CONSOLE

Build a comprehensive Forge Creator Console.

Include:

- Tenant directory
- Tenant creation
- Tenant suspension
- Tenant reactivation
- Product assignment
- Module entitlements
- Subscription plans
- Pricing
- Trials
- Renewals
- Grace periods
- Feature flags

- Usage limits
- Storage usage
- User limits
- Environment status
- Release management
- Template management
- Support access
- Security events
- Audit search
- Billing status
- GovCloud designation
- Data-isolation tier
- Customer health
- System announcements

The creator must not need direct database access for normal operations.

23. SUBSCRIPTIONS

Support:

- Monthly
- Annual
- Contract
- Trial
- Custom
- Government procurement
- Manual invoicing
- Purchase order
- Grace period
- Suspension
- Cancellation
- Reactivation

Do not immediately destroy access when a subscription expires.

Use stages:

```
ACTIVE
PAYMENT_DUE
GRACE_PERIOD
READ_ONLY
SUSPENDED
```

TERMINATED
ARCHIVED

Allow contract-specific emergency read access.

Never delete customer data solely because payment expired.

24. SUPPORT ACCESS

Support access must be:

- Time-limited
- Purpose-limited
- Tenant-approved where required
- Permission-limited
- MFA-protected
- Logged
- Visible in audit history

Support users must not automatically view:

- SSNs
- Medical information
- Disciplinary records
- Financial information
- Restricted exams

Create support sessions with:

```
support_user_id
tenant_id
approved_by
reason
scope
starts_at
expires_at
status
```

25. REPORTING

Create a shared report builder.

Support:

- Product
- Module
- Record type
- Selected fields
- Filters
- Groups
- Sorting
- Aggregations
- Calculations
- Saved reports
- Shared reports
- Scheduled reports
- PDF
- CSV
- XLSX

Sensitive fields must be permission filtered.

Dashboard widgets must use the same report/query framework where practical.

26. DOCUMENT GENERATION

Build a template-driven document engine.

Support:

- Certificates
- Transcripts
- Invoices
- Receipts
- Inspection reports
- Notices
- Hydrant damage reports
- Training records
- Rosters
- Attendance reports
- Apparatus reports
- Personnel reports

Templates must support:

- Tenant branding
- Headers

- Footers
- Logos
- Page numbers
- Dynamic fields
- Conditional sections
- Signatures
- QR verification
- Watermarks
- Versioning

Do not hard-code organization branding into templates.

27. NOTIFICATIONS

Support:

- In-app
- Email
- SMS abstraction
- Push abstraction

Create templates for:

- Enrollment
- Application status
- Waitlist
- Class reminders
- Attendance
- Exam results
- Certification expiration
- Invoice
- Payment
- Inspection assignment
- Violation
- Hydrant damage
- Work order
- Import completion
- Security event

Notifications must support:

- Tenant branding
- Variables
- Localization readiness
- Role targeting
- User preferences

- Delivery tracking
 - Retry
 - Failure status
 - Audit events
-

28. API STANDARDS

Use versioned APIs.

Example:

```
/api/v1/
```

Every endpoint must include:

- Authentication
- Tenant resolution
- Permission enforcement
- Input validation
- Output validation
- Correlation ID
- Structured logging
- Rate limiting where appropriate
- Error handling
- Audit event where required

Use consistent error format:

```
{
  "error": {
    "code": "PERMISSION_DENIED",
    "message": "You do not have permission to perform this action.",
    "correlationId": "uuid",
    "details": []
  }
}
```

Never expose:

- Stack traces
- SQL
- Secrets
- Internal infrastructure identifiers

- Sensitive personal data

Create OpenAPI documentation.

29. EVENT SYSTEM

Use domain events.

Examples:

```
PersonCreated
PersonMerged
StudentEnrolled
StudentWithdrawn
CoursePublished
ClassScheduled
ClassCompleted
CertificationIssued
CertificationExpiring
CertificateIssued
InvoiceCreated
PaymentRecorded
InspectionAssigned
ViolationCreated
HydrantDamageReported
ApparatusPlacedOutOfService
ImportCompleted
SensitiveDataViewed
```

Events must include:

```
interface DomainEvent<T> {
    eventId: string;
    eventType: string;
    eventVersion: number;
    tenantId: string;
    occurredAt: string;
    actorUserId?: string;
    correlationId: string;
    payload: T;
}
```

Use outbox pattern or another reliable delivery approach.

30. OBSERVABILITY

All services must emit structured JSON logs.

Include:

```
timestamp
level
service
environment
tenant_id
user_id
correlation_id
request_id
event_type
message
duration_ms
status
```

Do not log:

- SSN
- Password
- Tokens
- Secrets
- Full birth dates unless strictly required
- Financial account data
- Protected exam content

Create CloudWatch dashboards for:

- API errors
- Latency
- Database connections
- Queue depth
- Import failures
- Email failures
- Authentication failures
- Permission denials
- WAF blocks
- Storage growth
- Background job duration

Create actionable alarms.

31. SECURITY REQUIREMENTS

Implement:

- Least privilege IAM
- MFA
- Encryption at rest
- Encryption in transit
- KMS key separation
- Secrets Manager
- Security headers
- Content Security Policy
- CSRF protection
- XSS protection
- SQL injection protection
- Rate limiting
- Account lockout controls
- Session timeout
- Secure cookies
- Dependency scanning
- Secret scanning
- Container scanning
- SAST
- DAST where practical
- Audit logging
- Backup
- Recovery testing

Create:

```
/docs/security/threat-model.md  
/docs/security/data-classification.md  
/docs/security/access-control-matrix.md  
/docs/security/incident-response-plan.md  
/docs/security/backup-recovery-plan.md  
/docs/security/secure-development-standard.md  
/docs/security/sensitive-data-handling.md
```

32. DATA CLASSIFICATION

Classify data as:

PUBLIC
INTERNAL
CONFIDENTIAL
RESTRICTED

Examples:

Public

- Public course catalog
- Public certificate verification
- Public academy contact information

Internal

- Internal announcements
- General workflow configuration

Confidential

- Personnel contact information
- Training records
- Class records
- Department billing

Restricted

- SSNs
- Background checks
- Medical data
- Disciplinary records
- Banking data
- Restricted exams
- Security credentials

Apply controls based on classification.

33. MOBILE AND OFFLINE

Build mobile-first workflows for:

- Attendance
- Skills evaluations
- Inspections
- Preplans

- Apparatus checks
- Inventory checks
- Hydrant damage reports
- Incident reporting
- Signatures
- Photo capture
- Document upload

Offline support must include:

- Encrypted local data
- User scope
- Tenant scope
- Expiration
- Revocation
- Conflict detection
- Version checking
- Sync status
- Retry
- Manual conflict resolution

Do not provide unrestricted offline access to SSNs or other restricted records.

34. ACCESSIBILITY

Target WCAG 2.2 AA.

Require:

- Keyboard navigation
- Visible focus
- Semantic HTML
- Labels
- Screen-reader support
- Accessible errors
- Accessible dialogs
- Sufficient contrast
- Text scaling
- Captions
- Accessible PDF output
- No color-only statuses

Add automated accessibility testing and manual review requirements.

35. TESTING

Create:

- Unit tests
- Integration tests
- API tests
- Database tests
- RLS tests
- Authorization tests
- Import tests
- Export tests
- Workflow tests
- Document generation tests
- End-to-end tests
- Accessibility tests
- Security tests
- Performance tests
- Migration reconciliation tests

Critical test cases:

- Tenant A cannot read Tenant B data.
- Tenant A cannot write Tenant B data.
- Background jobs cannot cross tenant boundaries.
- S3 URLs cannot be used across tenants.
- Unauthorized user cannot view SSN.
- SSN access creates audit event.
- Deactivated dropdown values remain on historical records.
- Form versions remain renderable.
- Workflow versions preserve historical behavior.
- Duplicate imports do not silently create duplicate people.
- Failed imports can be reprocessed.
- Subscription suspension does not destroy data.
- Academy-to-RMS sync shares only authorized fields.
- GovCloud configuration does not require commercial ARN assumptions.

Set minimum coverage targets but do not optimize only for percentage.

36. CI/CD

Create GitHub Actions workflows for:

- Install
- Lint

- Typecheck
- Unit tests
- Integration tests
- Build
- Dependency scan
- Secret scan
- Infrastructure validation
- CDK synth
- Container build
- Container scan
- ECR push
- Deployment
- Database migration
- Smoke tests
- Rollback

Use environment protection rules.

Production deployment must require approval.

Never deploy directly from a developer laptop.

Use immutable container tags based on commit SHA.

37. DATABASE MIGRATIONS

All schema changes must use versioned migrations.

Migration rules:

- Backward-compatible where possible
- Tested on representative data
- Reviewed
- Reversible where practical
- Logged
- Included in deployment workflow
- Protected against concurrent execution

Do not manually alter production schemas.

38. FIREBASE MIGRATION

38.1 Inventory

Map all:

- Collections
- Subcollections
- Documents
- Authentication accounts
- Storage objects
- Cloud Functions
- Scheduled jobs
- Rules
- Integrations

38.2 Preserve source identifiers

Store:

```
source_system  
source_project  
source_collection  
source_document_id  
source_path  
migration_batch_id
```

38.3 Migration pipeline

1. Export Firebase data.
2. Export Firebase users.
3. Export storage metadata.
4. Copy approved files.
5. Transform records.
6. Validate relationships.
7. Match people.
8. Import reference data.
9. Import organizations.
10. Import people.
11. Import product records.
12. Import documents.
13. Reconcile counts.
14. Reconcile hashes.
15. Reconcile relationships.

16. Generate exception report.
17. Obtain approval.
18. Run final delta migration.
19. Place Firebase module read-only.
20. Redirect users.
21. Preserve approved archive.

Do not delete Firebase data until formal acceptance and retention approval.

39. GOVCLLOUD PREPARATION

Create:

```
/docs/govcloud/service-compatibility-register.md
/docs/govcloud/deployment-plan.md
/docs/govcloud/data-transfer-plan.md
/docs/govcloud/partition-differences.md
/docs/govcloud/operational-separation.md
```

For each AWS service, track:

```
Service
Commercial availability
GovCloud availability
Region
Feature differences
Fallback
Abstraction required
Review date
Approved status
```

GovCloud must deploy from the same source code, but with:

- Separate pipeline
- Separate accounts
- Separate credentials
- Separate secrets
- Separate databases
- Separate storage
- Separate logs
- Separate keys
- Separate domains
- Separate monitoring

Do not assume commercial database snapshots can be copied into GovCloud.

Create export/import migration bundles containing:

- Manifest
- Schema version
- Record counts
- File inventory
- SHA-256 hashes
- Encryption metadata
- Migration batch ID
- Source environment
- Destination environment
- Validation results

40. DISASTER RECOVERY

Define:

- RPO
- RTO
- Backup schedule
- Backup retention
- Database restore procedure
- S3 recovery
- Infrastructure recreation
- DNS failover
- Secret recovery
- Key recovery
- Incident communication
- Recovery testing schedule

Create automated backup verification.

A backup is not considered valid until restore testing succeeds.

41. REQUIRED DOCUMENTATION

Maintain:

`README.md`
`/docs/architecture/system-overview.md`

```
/docs/architecture/tenant-isolation.md
/docs/architecture/event-architecture.md
/docs/architecture/aws-architecture.md
/docs/architecture/govcloud-readiness.md

/docs/api/openapi.yaml
/docs/api/authentication.md
/docs/api/authorization.md
/docs/api/errors.md

/docs/data-model/core.md
/docs/data-model/academy.md
/docs/data-model/rms.md

/docs/deployment/local-development.md
/docs/deployment/development.md
/docs/deployment/staging.md
/docs/deployment/production.md
/docs/deployment/govcloud.md

/docs/migration/firebase.md
/docs/migration/validation.md
/docs/migration/rollback.md

/docs/operations/runbook.md
/docs/operations/monitoring.md
/docs/operations/backup.md
/docs/operations/incident-response.md

/docs/user-guides/creator-console.md
/docs/user-guides/configuration-studio.md
/docs/user-guides/import-center.md
/docs/user-guides/academy-admin.md
/docs/user-guides/rms-admin.md
```

Documentation must be updated with code changes.

42. IMPLEMENTATION PHASES

Phase 0: Discovery

Deliver:

- Existing system inventory

- Firebase inventory
- Feature matrix
- Data model inventory
- Security assessment
- GovCloud compatibility register
- Architecture decisions

Phase 1: Monorepo and developer platform

Deliver:

- Monorepo
- Shared packages
- Local Docker environment
- Formatting
- Linting
- Testing
- CI
- Base documentation

Phase 2: AWS landing zone

Deliver:

- CDK infrastructure
- Networking
- Logging
- Security controls
- Development deployment
- Database
- S3
- Cognito
- ECS
- ECR
- Queues
- Monitoring

Phase 3: Shared platform services

Deliver:

- Tenant service
- Identity service
- Person service
- Authorization
- Audit
- Documents

- Notifications
- Subscription entitlements
- Feature flags

Phase 4: Configuration platform

Deliver:

- Dropdown manager
- Custom field manager
- Form builder
- Workflow builder
- Terminology
- Branding
- Navigation editor
- Role builder

Phase 5: Import and export platform

Deliver:

- Templates
- Upload
- Mapping
- Validation
- Duplicate detection
- Preview
- Background processing
- Error handling
- Reprocessing
- Export
- Audit

Phase 6: Forge Academy core

Deliver:

- Academy administration
- People
- Students
- Instructors
- Courses
- Classes
- Enrollment
- Attendance
- Certifications
- Department portal

PHASE 5A: SHARED QR AND PUBLIC ACCESS PLATFORM

Deliver:

- Shared product-neutral QR core package
- Dynamic public token resolver
- QR link lifecycle and versioning
- Document and procedure destination integration
- SVG and PNG generation with decode verification
- Printable PDF labels and batch generation
- Public access policies and controlled anonymous delivery
- Archive, restore, disable, revoke, expiration, and soft deletion
- Scan analytics, audit events, security events, and monitoring
- Creator Console and tenant administration interfaces
- Forge Industrial first-consumer adapters
- Product adapter contracts for later Forge RMS and Forge Academy integration

Sequencing rule

Phase 5A may be developed in Forge Industrial first when operationally necessary, but it must remain a shared Forge capability. It must not interrupt an active migration, security, or import-platform acceptance gate. RMS and Academy adapters require separate authorization.

Initial sprint sequence

- QR-S0 - Architecture, repository assessment, adapter boundaries, threat model, and implementation plan
- QR-S1 - Shared schema, migration, RLS, permissions, and audit events
- QR-S2 - Management CRUD APIs and lifecycle controls
- QR-S3 - Document storage, versioning, and secure public resolver
- QR-S4 - QR image, label, and batch generation
- QR-S5 - Forge Industrial management UI and initial adapters
- QR-S6 - Analytics, monitoring, and operational hardening
- QR-S7 - RMS and Academy adapters after separate approval
- QR-S8 - Security, performance, accessibility, and production-readiness review

Phase 7: Forge Academy advanced

Deliver:

- LMS
- Testing
- Skills
- Certificates
- Transcripts
- Dormitories
- Finance
- Reporting
- Public registration

Phase 8: Academy migration

Deliver:

- Firebase extraction
- Transformations
- Trial migration
- Reconciliation
- Final migration
- Read-only legacy archive

Phase 9: Forge RMS core

Deliver:

- Department administration
- Personnel
- Training
- Scheduling
- Apparatus
- Inventory
- Documents
- Academy integration

Phase 10: Forge RMS operations

Deliver:

- Prevention
- Inspections
- Occupancies
- Preplans

- Hydrants
- Incident reporting
- EMS operations
- Reporting

Phase 11: Hardening

Deliver:

- Security testing
- Performance testing
- Accessibility testing
- Backup testing
- Recovery testing
- Penetration-test remediation
- Operational runbooks

Phase 12: GovCloud readiness

Deliver:

- GovCloud deployment templates
- GovCloud service verification
- Partition testing
- Data-transfer tooling
- Separate pipeline
- GovCloud staging deployment after approval

43. CURSOR WORKING RULES

Cursor must follow these rules throughout development:

1. Inspect existing code before replacing it.
2. Do not invent existing behavior without checking.
3. Preserve valuable current features.
4. Do not create placeholder-only modules.
5. Do not leave buttons without actions.
6. Do not create fake statistics in production screens.
7. Do not hard-code tenant values.
8. Do not hard-code dropdowns that should be configurable.
9. Do not use blank text fields where controlled dropdowns are appropriate.
10. Do not expose tenant IDs without authorization checks.
11. Do not trust frontend validation.
12. Do not bypass database RLS.
13. Do not log sensitive data.

14. Do not store secrets in source code.
 15. Do not create manual-only AWS production resources.
 16. Do not deploy untested database migrations.
 17. Do not remove Firebase features before replacement acceptance.
 18. Do not merge large changes without tests and documentation.
 19. Do not use SSN as an identifier.
 20. Do not assume GovCloud behaves exactly like commercial AWS.
 21. Do not add dependencies without security and GovCloud review.
 22. Do not declare a module complete based only on UI completion.
 23. Keep each implementation phase deployable.
 24. Keep the application build passing.
 25. Resolve TypeScript and lint errors rather than suppressing them.
 26. Use feature flags for incomplete production features.
 27. Create migration notes for every breaking change.
 28. Create an architecture decision record for major technical decisions.
 29. Use synthetic data in development.
 30. Maintain auditability across all sensitive actions.
-

44. DEFINITION OF DONE FOR EVERY MODULE

A module is complete only when it includes:

- Working UI
- Working API
- Database schema
- Database migrations
- Tenant isolation
- Permissions
- Audit events
- Validation
- Error handling
- Loading states
- Empty states
- Search
- Filters
- Pagination
- Sort
- Mobile layout
- Accessibility
- Import support where applicable
- Export support where applicable
- Reporting support
- Configuration support
- Automated tests
- Documentation

- Monitoring
- Migration support
- Security review

No module may be marked complete while containing:

- Placeholder buttons
 - Mock-only data
 - Unprotected routes
 - Cross-tenant exposure
 - Hard-coded tenant values
 - Unvalidated inputs
 - Unlogged sensitive actions
 - Broken mobile layouts
 - Unresolved critical errors
-

45. FIRST DEVELOPMENT SPRINT

Begin with these tasks only:

Sprint 1A: Discovery and inventory

- Inspect the current Forge Academy repository.
- Inspect the current Forge RMS repository.
- Document every route and module.
- Document every Firestore collection.
- Document every Firebase Storage path.
- Document every Cloud Function.
- Document authentication and roles.
- Create the feature preservation matrix.
- Create the migration inventory.
- Identify sensitive fields.
- Identify hard-coded configuration.

Sprint 1B: Architecture foundation

- Create the monorepo.
- Configure pnpm and Turborepo.
- Configure TypeScript strict mode.
- Configure ESLint and Prettier.
- Configure shared types.
- Configure test framework.
- Configure Docker Compose for local PostgreSQL.
- Create basic CI.
- Create ADR templates.

- Create environment schema validation.

Sprint 1C: AWS CDK baseline

Create development infrastructure for:

- VPC
- Public and private subnets
- Security groups
- ECS cluster
- ECR repositories
- Aurora PostgreSQL
- Secrets Manager
- KMS
- S3 document bucket
- S3 import bucket
- Cognito
- SQS
- CloudWatch
- CloudTrail integration
- WAF baseline
- Backup baseline

Sprint 1D: Core platform skeleton

Create:

- Tenant model
- Organization model
- User model
- Person model
- Role model
- Permission model
- Audit model
- Product entitlement model
- Basic authentication
- Tenant resolution
- RLS policies
- Cross-tenant security tests

Do not begin rebuilding advanced Academy or RMS screens until these foundations are operating.

46. REQUIRED SPRINT OUTPUT FORMAT

At the end of every sprint, generate:

```
/docs/sprints/SPRINT-XX-summary.md
```

Include:

- Objectives
- Completed work
- Files created
- Files modified
- Database migrations
- Infrastructure changes
- Tests added
- Test results
- Security considerations
- Known issues
- Deferred work
- Deployment status
- Rollback notes
- Next recommended sprint

Also update:

```
/docs/project-status.md
```

Use statuses:

```
NOT_STARTED  
IN_DISCOVERY  
IN_PROGRESS  
BLOCKED  
READY_FOR_REVIEW  
COMPLETE  
DEFERRED
```

47. AWS DEPLOYMENT COMMANDS

Create documented commands similar to:

```
pnpm install  
pnpm lint  
pnpm typecheck  
pnpm test
```

```
pnpm build
pnpm test:e2e
pnpm infra:synth
pnpm infra:diff:development
pnpm infra:deploy:development
```

Create environment-specific deployment scripts.

Do not embed credentials in scripts.

Use AWS profiles or CI identity federation.

48. FINAL PLATFORM OUTCOME

The final result must allow Forge Public Safety to:

- Operate Forge Academy and Forge RMS on one secure foundation
- Keep product modules independently licensed
- Allow tenants to customize their own environments
- Allow tenants to manage dropdowns
- Allow tenants to build forms
- Allow tenants to build workflows
- Allow tenants to create custom fields
- Allow imports for every major category
- Allow secure exports
- Protect SSNs and FEMA SID values
- Maintain tenant isolation
- Share approved data between Academy and RMS
- Avoid creator involvement in routine tenant configuration
- Deploy to commercial AWS
- Deploy a separate future GovCloud environment
- Migrate existing Firebase data safely
- Expand into future Forge products without another full rebuild

This platform must be engineered as a long-term enterprise system, not as a short-lived migration project.